三目並べ：9つの useState を配列1つにまとめた説明

このPDFは、App.tsx
でマスの状態管理が大きく変わった理由と、前後の違いを初心者向けにまとめたものです。

1. 何が変わったか（一言）

前は「マスごとに変数が9個」でした。今は「マス9個分を配列1つ」で持つ形です。見た目のゲームのルール（○と×を交互に置く）は同じです。変わったのは、データの持ち方とクリック処理の書き方です。

2. 変更前：変数が9個

それぞれのマスに、自分用の useState がありました。

```
const [square1, setSquare1] = useState("1")
const [square2, setSquare2] = useState("2")
...
const [square9, setSquare9] = useState("9")
```

クリック処理も、マスの数だけほぼ同じ関数が並んでいました（handleClick1 〜 handleClick9）。1マス分のルールを直すとき、同じ修正を9回する必要がありました。

3. 変更後：配列が1つ

9マスを1本の配列に入れます。番号は 0 から始まります。

```
// 位置の対応
// 0 1 2
// 3 4 5
// 6 7 8

const [squares, setSquares] = useState([
  "1", "2", "3",
  "4", "5", "6",
  "7", "8", "9",
])
```

例: squares[0] が左上、squares[4] が中央、squares[8] が右下です。

4. クリック処理も1つに

変更前は「何番のボタンか」ごとに関数を分けていました。変更後は、押された番号 index を受け取って、同じ関数で処理します。

```
function handleClick(index: number) {
  // すでに ○ / ✕ なら何もしない
  if (squares[index] === "○" || squares[index] === "✕") {
    return
  }

  // 配列をコピーしてから書き換える
  const nextSquares = squares.slice()
  nextSquares[index] = isOTurn ? "○" : "✕"
  setSquares(nextSquares)

  // 次の人の番へ
  setIsOTurn(!isOTurn)
}
```

ボタン側では、何番かを渡すだけです。

```
<Square value={squares[0]} onClick={() => handleClick(0)} />
<Square value={squares[4]} onClick={() => handleClick(4)} />
```

5. なぜ配列にコピーするのか

React では、いま使っている配列をその場で書き換えるより、「新しい配列」を渡す方が安全です。

```
// 悪い例（元の配列を直接いじる）
squares[index] = "○"
setSquares(squares)

// 良い例（コピーを作ってから変える）
const nextSquares = squares.slice()
nextSquares[index] = "○"
setSquares(nextSquares)
```

slice() は配列のコピーを作ります。コピーの方だけ書き換えて setSquares に渡します。

6. 前後の対応表

- square1 → squares[0]
- square2 → squares[1]
- square3 → squares[2]
- square4 → squares[3]
- square5 → squares[4]
- square6 → squares[5]
- square7 → squares[6]
- square8 → squares[7]

・ square9 → squares[8]
・ setSquare1(...) など9個 → setSquares(nextSquares) 1回
・ handleClick1 など9個 → handleClick(index) 1個

7. まとめ

・ ゲームの遊び方は同じ (交互に ○ / ×)
・ データの持ち方が「9変数」から「配列1つ」へ
・ クリック処理が「9関数」から「番号付きの1関数」へ
・ この形だと、あとから勝敗判定や履歴を足しやすくなる

ファイルの場所: src/App.tsx